

✓ Notebook 02 — Data Cleaning & Preprocessing

Decodelabs Internship | Week 1 | Task 2

I cleaned the raw Heart Disease dataset in this notebook. This includes handling missing values, fixing data types, removing any problematic rows, and creating a clean, analysis-ready version of the data.

✓ Importing libraries and set paths

```
import os
import pandas as pd
import numpy as np

# File paths
NOTEBOOK_DIR = os.getcwd()
PROJECT_ROOT = os.path.dirname(NOTEBOOK_DIR)
RAW_FILE = os.path.join(PROJECT_ROOT, "data", "raw", "heart_cleveland_raw.csv")
CLEAN_FILE = os.path.join(PROJECT_ROOT, "data", "processed", "heart_cleveland_clean.csv")
TABLES_DIR = os.path.join(PROJECT_ROOT, "reports", "tables")
os.makedirs(TABLES_DIR, exist_ok=True)

COLUMN_NAMES = [
    "age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
    "thalach", "exang", "oldpeak", "slope", "ca", "thal", "target"
]

print("Libraries and paths configured.")
print(f"Raw file : {RAW_FILE}")
print(f"Clean file: {CLEAN_FILE}")
```

```
Libraries and paths configured.
Raw file : c:\Users\Peter\Documents\EXTRA-CURRICULA\Internship\Decodelab\heart_cleveland_raw.csv
Clean file: c:\Users\Peter\Documents\EXTRA-CURRICULA\Internship\Decodelab\heart_cleveland_clean.csv
```

✓ Loading the raw data

```
df = pd.read_csv(RAW_FILE, header=None, names=COLUMN_NAMES, na_values='?')

print(f"Loaded dataset: {df.shape[0]} rows × {df.shape[1]} columns")
print()
df.head()
```

Loaded dataset: 303 rows × 14 columns

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0

✓ Inspecting missing values in detail

```
# I confirm again exactly which columns have missing values and how many.  
# I check this BEFORE making any changes so I have a baseline to compare to.
```

```
print("=== Missing Values (Before Cleaning) ===")  
missing = df.isnull().sum()  
missing_pct = (missing / len(df) * 100).round(2)  
  
for col in df.columns:  
    if missing[col] > 0:  
        print(f" {col:12s} : {missing[col]} missing ({missing_pct[col]}%")  
  
print()  
print(f"Total missing values: {missing.sum()} out of {df.size} cells "  
      f"({missing.sum()/df.size*100:.2f}%")
```

```
=== Missing Values (Before Cleaning) ===  
ca          : 4 missing (1.32%)  
thal        : 2 missing (0.66%)  
  
Total missing values: 6 out of 4242 cells (0.14%)
```

```
# Atual rows that have missing values.  
# With only 303 rows, I can afford to look at these individually.
```

```
rows_with_missing = df[df.isnull().any(axis=1)]  
print(f"Rows with at least one missing value: {len(rows_with_missing)}")  
print()  
print(rows_with_missing)
```

```
Rows with at least one missing value: 6
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak
87	53.0	0.0	3.0	128.0	216.0	0.0	2.0	115.0	0.0	0.0

166	52.0	1.0	3.0	138.0	223.0	0.0	0.0	169.0	0.0	0.0
192	43.0	1.0	4.0	132.0	247.0	1.0	2.0	143.0	1.0	0.1
266	52.0	1.0	4.0	128.0	204.0	1.0	0.0	156.0	1.0	1.0
287	58.0	1.0	2.0	125.0	220.0	0.0	0.0	144.0	0.0	0.4
302	38.0	1.0	3.0	138.0	175.0	0.0	0.0	173.0	0.0	0.0

	slope	ca	thal	target
87	1.0	0.0	NaN	0
166	1.0	NaN	3.0	0
192	2.0	NaN	7.0	1
266	2.0	0.0	NaN	2
287	2.0	NaN	7.0	0
302	1.0	NaN	3.0	0

✓ Handling missing values

I have 6 rows with missing values in `ca` (4 rows) and `thal` (2 rows).

Decision: I will drop these 6 rows.

Why dropping is acceptable here:

- 6 rows out of 303 is only 2% of the data, the loss is minimal.
- The `ca` and `thal` columns are clinically important features; filling them with the mean or mode could introduce misleading patterns.
- For a dataset this small, it is better to have a slightly smaller but fully complete dataset than to impute values for a clinically complex feature.

```
# axis=0 means "drop rows" (axis=1 would drop columns).
```

```
df_clean = df.dropna(axis=0)
```

```
print(f"Rows before dropping missing: {len(df)}")
print(f"Rows after dropping missing: {len(df_clean)}")
print(f"Rows removed: {len(df) - len(df_clean)}")
print()
```

```
# Verify no missing values remain
remaining_missing = df_clean.isnull().sum().sum()
print(f"Missing values remaining: {remaining_missing}")
```

```
Rows before dropping missing: 303
Rows after dropping missing: 297
Rows removed: 6
```

```
Missing values remaining: 0
```

✓ Fixing data types

```
# After dropping missing rows, I checked the data types.
# Some columns that pandas loaded as float64 should be integers (because the

print("=== Data types before fixing ===")
print(df_clean.dtypes)
```

```
=== Data types before fixing ===
age          float64
sex          float64
cp           float64
trestbps     float64
chol         float64
fbs          float64
restecg      float64
thalach      float64
exang        float64
oldpeak      float64
slope        float64
ca           float64
thal         float64
target       int64
dtype: object
```

```
# I converted columns that should be integers.

integer_columns = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "t

for col in integer_columns:
    df_clean[col] = df_clean[col].astype(int)

print("=== Data types after fixing ===")
print(df_clean.dtypes)
```

```
=== Data types after fixing ===
age          float64
sex          int64
cp           int64
trestbps     float64
chol         float64
fbs          int64
restecg      int64
thalach      float64
exang        int64
oldpeak      float64
slope        int64
ca           int64
thal         int64
target       int64
dtype: object
```

✓ Binarising the target variable

```
# For binary classification, I converted the target variable (0,1,2,3,4) to:
# 0 = no heart disease
# 1 = heart disease present (any severity)
#
# I check the original distribution first.

print("=== Original target distribution ===")
print(df_clean["target"].value_counts().sort_index())
print()
```

```
=== Original target distribution ===
target
0      160
1       54
2       35
3       35
4       13
Name: count, dtype: int64
```

```
# New binary target column created.

df_clean["target"] = (df_clean["target"] > 0).astype(int)

print("=== Binarised target distribution ===")
print(df_clean["target"].value_counts().sort_index())
print()

no_disease = (df_clean["target"] == 0).sum()
disease     = (df_clean["target"] == 1).sum()
total       = len(df_clean)

print(f"No heart disease (0) : {no_disease} patients ({no_disease/total*100:.1f}%")
print(f"Heart disease      (1) : {disease}    patients ({disease/total*100:.1f}%")
```

```
=== Binarised target distribution ===
target
0      160
1      137
Name: count, dtype: int64

No heart disease (0) : 160 patients (53.9%)
Heart disease      (1) : 137    patients (46.1%)
```

✓ Checking for duplicate rows

```
# I still try to confirm if there are any duplicate rows in the cleaned data

n_duplicates = df_clean.duplicated().sum()
print(f"Duplicate rows found: {n_duplicates}")

if n_duplicates > 0:
    print("Showing duplicate rows:")
    print(df_clean[df_clean.duplicated(keep=False)])
else:
    print("No duplicate rows. The dataset is clean.")
```

```
Duplicate rows found: 0
No duplicate rows. The dataset is clean.
```

✓ Final validation check

```
# Before saving, I do a final check to make sure the cleaned dataset looks r

print("=== Final Clean Dataset Summary ===")
print(f"Shape   : {df_clean.shape[0]} rows × {df_clean.shape[1]} columns")
print(f"Missing : {df_clean.isnull().sum().sum()} values")
print(f"Dupl.    : {df_clean.duplicated().sum()}")
print()
print("Column dtypes:")
print(df_clean.dtypes)
print()
print("First 5 rows:")
df_clean.head()
```

```
=== Final Clean Dataset Summary ===
Shape   : 297 rows × 14 columns
Missing : 0 values
Dupl.    : 0
```

```
Column dtypes:
age          float64
sex          int64
cp          int64
trestbps     float64
chol         float64
fbs          int64
restecg      int64
thalach      float64
exang        int64
oldpeak      float64
slope        int64
ca           int64
thal         int64
target       int64
dtypes: object
```

```
dtype: object
```

First 5 rows:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	63.0	1	1	145.0	233.0	1	2	150.0	0	2.3	3
1	67.0	1	4	160.0	286.0	0	2	108.0	1	1.5	2
2	67.0	1	4	120.0	229.0	0	2	129.0	1	2.6	2
3	37.0	1	3	130.0	250.0	0	0	187.0	0	3.5	3
4	41.0	0	2	130.0	204.0	0	2	172.0	0	1.4	1

```
print("=== Descriptive Statistics (Clean Dataset) ===")
df_clean.describe().round(2)
```

```
=== Descriptive Statistics (Clean Dataset) ===
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang
count	297.00	297.00	297.00	297.00	297.00	297.00	297.00	297.00	297.00
mean	54.54	0.68	3.16	131.69	247.35	0.14	1.00	149.60	0.33
std	9.05	0.47	0.96	17.76	52.00	0.35	0.99	22.94	0.47
min	29.00	0.00	1.00	94.00	126.00	0.00	0.00	71.00	0.00
25%	48.00	0.00	3.00	120.00	211.00	0.00	0.00	133.00	0.00
50%	56.00	1.00	3.00	130.00	243.00	0.00	1.00	153.00	0.00
75%	61.00	1.00	4.00	140.00	276.00	0.00	2.00	166.00	1.00
max	77.00	1.00	4.00	200.00	564.00	1.00	2.00	202.00	1.00

✓ Cleaned dataset saved

```
df_clean.to_csv(CLEAN_FILE, index=False)
print(f"Clean dataset saved to: {CLEAN_FILE}")
print(f"Rows saved: {len(df_clean)}")
print(f"Columns: {list(df_clean.columns)}")
```

```
Clean dataset saved to: c:\Users\Peter\Documents\EXTRA-CURRICULA\Internship\D
Rows saved: 297
Columns: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach'
```

```
# I also save a summary table for the report.
```

```
summary_table = df_clean.describe().round(2)
```

```
summary_path = os.path.join(TABLES_DIR, "descriptive_statistics.csv")
summary_table.to_csv(summary_path)
print(f"Summary statistics table saved to: {summary_path}")
```

Summary statistics table saved to: c:\Users\Peter\Documents\EXTRA-CURRICULA\I

Summary of Notebook 02

I cleaned the raw dataset and produced a clean, analysis-ready file.

Changes made:

- Dropped **6 rows** with missing values in `ca` and `tha1` columns (2% of data).
- Converted 9 columns from `float64` to `int64` for correct categorical handling.
- Binarised the `target` column: values 1–4 → 1 (disease present), 0 → 0 (no disease).
- Confirmed **0 duplicate rows**.

Final clean dataset: 297 rows × 14 columns, no missing values.

Next step: Notebook 03 — Exploratory Data Analysis